

Chapter 1: The Architecture of Large Language Models

Large Language Models (LLMs) are built upon the Transformer architecture, first introduced in 2017. The core innovation of the Transformer is the self-attention mechanism, which allows the model to weigh the relevance of different words in a sentence relative to each other, regardless of their distance. During pre-training, models are exposed to massive corpuses of text, learning statistical relationships between tokens by predicting missing words.

This training allows models to acquire broad knowledge and capture syntactic and semantic patterns. Parameters within these neural networks represent weights that adjust during training via backpropagation. However, once training concludes, the model weights are frozen, meaning the model's static base knowledge is capped.

Chapter 2: Hallucinations and Context Limitations

A major limitation of pre-trained LLMs is their tendency to 'hallucinate'—generating facts or numbers that sound plausible but are entirely incorrect. Because these models predict subsequent words based on probability rather than a database of truth, they cannot separate training-set bias from factual accuracy when queried on unseen topics.

Furthermore, every language model is restricted by its context window size. Early models were restricted to just 512 tokens, meaning they could not read or remember long conversations. While modern context limits have expanded to thousands or millions of tokens, sending long documents leads to high processing fees, increased latency, and a drop in recall quality known as the 'lost in the middle' phenomenon.

Chapter 3: Foundations of Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) solves these issues by acting as a bridge between the user's question and private, dynamic data sources. Instead of relying on static training weights to answer, the RAG system first searches external indices for documents relevant to the query, retrieves matching fragments, and injects them into the prompt.

By doing so, the LLM is transformed from an information storage engine into a text synthesis engine. The injected document chunks serve as the 'ground truth' reference. If the retrieved context is correct, the LLM can generate highly accurate, grounded answers, substantially eliminating hallucinations and resolving knowledge cutoff restrictions.

Chapter 4: Text Chunking and Embedding Vectorization

For RAG to work, documents must be divided into smaller paragraphs called 'chunks'. This chunking is critical: if chunks are too large, they overflow the LLM's context window; if too small, they lose key surrounding context. Common strategies include Fixed-Size chunking, Semantic chunking (sentence splits based on meaning breaks), and Hierarchical chunking.

Once chunked, the text segments are converted into dense vector embeddings using encoder models like 'all-MiniLM-L6-v2'. This model maps strings to a 384-dimensional mathematical space. Words or sentences with similar conceptual meanings are placed close together in this multi-dimensional space, allowing mathematical comparisons between queries and context.

Chapter 5: Vector Databases and ChromaDB Indexing

Vector databases like ChromaDB are designed to store, manage, and query dense vector representations. Unlike traditional SQL databases that match exact strings, vector databases calculate the distance between coordinates using metrics like Cosine Similarity or Euclidean Distance to return the top semantically relevant context chunks.

To speed up vector comparisons, these databases construct indexing topologies such as Hierarchical Navigable Small World (HNSW) graphs. When a user asks a question, the query is embedded, compared against the HNSW index, and matching text chunks are returned instantly. This enables real-time search lookup across millions of reference pages.